

ESP32 Digital Signature Benchmark

Phase A: Isolated Micro-Benchmarks (No Network Involved)

FYP Documentation: Embedded VPN Client

October 1, 2026

1 Phase A: Isolated Micro-Benchmarks (No Network Involved)

Phase A benchmarks the raw cryptographic execution of three signature schemes on an ESP32-D0WDQ6 microcontroller running at 240 MHz. Tests run locally in memory without network stack overhead to profile pure computational cost, memory allocation, and payload footprint:

1. **Ed25519**: Modern Edwards-curve signature algorithm (Curve25519).
2. **ECDSA P-256**: Standard NIST elliptic curve algorithm.
3. **RSA-3072**: Classic RSA algorithm configured at equivalent 128-bit security.

2 Key Performance Indicators (KPIs)

The evaluation profiles 10 Key Performance Indicators (KPIs) covering execution latency, hardware workload, wire footprint, and RAM consumption.

2.1 KPI Definitions and Operational Impact

- **SIG-1 (Key Generation Time)**: Elapsed time to generate a fresh public and private key pair. Determines the latency overhead when creating ephemeral keys for session forward secrecy.
- **SIG-2 (Signing Time)**: Elapsed time to compute a digital signature over a 64-byte challenge payload. Governs client responsiveness during mutual authentication handshakes.
- **SIG-3 (Verification Time)**: Elapsed time to verify an incoming digital signature from the VPN gateway.
- **SIG-4, SIG-5, SIG-6 (CPU Cycles)**: Hardware processor cycles consumed during key generation, signing, and verification. Because the ESP32 clock operates at 240 MHz (240,000,000 cycles per second), hardware cycle counts provide an exact, clock-invariant metric:

$$\text{Time (ms)} = \frac{\text{CPU Cycles}}{240,000}$$

- **SIG-7 (Public Key Size)**: Encoded byte length of the public key transmitted over the communication channel.
- **SIG-8 (Signature Size)**: Encoded byte length of the signature payload appended to network packets.
- **Total Handshake Overhead (Key + Signature)**: Aggregate authentication data transmitted per handshake round ($S_{\text{pub}} + S_{\text{sig}}$).

- **SIG-9 (Heap Memory Difference):** Dynamic RAM allocated from the ESP32 heap during cryptographic operations. Excessive allocations risk Out-Of-Memory (OOM) resets.
- **SIG-10 (Flash Footprint):** Non-volatile program memory occupied by the compiled binary on flash storage (436,560 bytes, 33% of 1.31 MB).

2.2 Iteration Count Strategy

- **Fast Benchmarks ($N = 100$):** Ed25519 (all operations), ECDSA (all operations), and RSA-3072 (sign and verify) were executed 100 times to compute stable arithmetic means and cancel out OS scheduling jitter.
- **RSA Key Generation ($N = 10$):** RSA-3072 key generation relies on stochastic prime searching, taking between 2 and 16 seconds per attempt on the ESP32. Iterations were capped at 10 to prevent Task Watchdog Timer (TWDT) timeouts while capturing representative variance.

Table 1: Summary of Benchmarking KPIs

KPI ID	Metric Name	Description
SIG-1	Key Generation Time	Time (ms) to create public and private key pair
SIG-2	Signing Time	Time (ms) to sign a 64-byte challenge message
SIG-3	Verification Time	Time (ms) to verify a received signature
SIG-4	KeyGen CPU Cycles	Hardware clock cycles spent creating keys
SIG-5	Signing CPU Cycles	Hardware clock cycles spent signing
SIG-6	Verification CPU Cycles	Hardware clock cycles spent verifying
SIG-7	Public Key Size	Size of public key in bytes
SIG-8	Signature Size	Size of signature in bytes
SIG-9	Heap Memory Used	Dynamic RAM consumed during test in bytes
SIG-10	Flash Footprint	Program storage space used on flash (bytes)

3 Comparison Tables

3.1 Performance Comparison

Table 2 summarizes the empirical metrics collected across all three algorithms, indicating the best performer for each parameter.

3.2 RSA-3072 Key Generation Distribution

Because RSA prime generation is non-deterministic, execution times vary significantly across runs. Table 3 lists the latencies recorded for all 10 individual key generation iterations.

Run 8 completed in 210 ms when candidate integers quickly satisfied Miller-Rabin primality tests, whereas Run 10 required 16,231 ms (16.2 seconds) due to repeated prime candidate rejections. This represents a 77-fold variance between minimum and maximum execution latency.

4 Selection Rationale for Ed25519

Ed25519 was selected as the digital signature scheme for the ESP32 VPN client based on four critical technical advantages:

Table 2: Performance Comparison: Ed25519 vs. ECDSA P-256 vs. RSA-3072

Metric	Ed25519	ECDSA P-256	RSA-3072	Winner	Performance Factor
<i>Execution Latency (Milliseconds)</i>					
Key Generation Time	8.49 ms	145.94 ms	6,139.23 ms	Ed25519	17.2× faster than ECDSA
Signing Time	9.35 ms	157.76 ms	370.13 ms	Ed25519	16.9× faster than ECDSA
Verification Time	19.77 ms	312.14 ms	1.66 ms	RSA-3072	RSA verify is 11.9× faster
<i>Hardware CPU Cycles (@ 240 MHz)</i>					
KeyGen Cycles	2,038,136	35,025,008	1,473,414,175	Ed25519	Lowest compute workload
Signing Cycles	2,244,252	37,861,518	88,830,344	Ed25519	Lowest compute workload
Verification Cycles	4,744,893	74,912,688	399,080	RSA-3072	Lowest verify cycles
<i>Communication Payload (Bytes)</i>					
Public Key Size	32 bytes	65 bytes	384 bytes	Ed25519	12.0× smaller than RSA
Signature Size	64 bytes	70 bytes	384 bytes	Ed25519	6.0× smaller than RSA
Total Handshake Payload	96 bytes	135 bytes	768 bytes	Ed25519	8.0× smaller than RSA
<i>Memory and Storage Utilization</i>					
Heap Memory Consumed	108 bytes	408 bytes	4,732 bytes	Ed25519	43.8× lower RAM footprint
Flash Footprint	436,560 bytes (33% of 1.31 MB)			Shared Binary	Common crypto core

Table 3: RSA-3072 Key Generation Latencies across 10 Iterations

Iteration	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	Mean
Latency (ms)	5,095	6,125	4,887	4,956	13,736	1,175	5,922	210	3,056	16,231	6,139.23

- Minimal Computation Latency (17x speed advantage):** Ed25519 completes key generation in 8.49 ms and signing in 9.35 ms. In comparison, ECDSA requires approximately 150 ms for each operation, and RSA requires 370 ms to sign. The entire Ed25519 client authentication step executes in under 20 ms, preventing connection setup delays.
- Compact Wire Payload (8x smaller than RSA):** Ed25519 requires only 96 bytes total on the wire (32-byte public key and 64-byte signature), compared to 135 bytes for ECDSA and 768 bytes for RSA-3072. The compact packet footprint avoids MTU fragmentation and reduces radio transmission time, directly preserving battery power.
- Low Heap Memory Overhead (108 bytes vs 4,732 bytes):** The ESP32 has approximately 320 KB of usable RAM shared among Wi-Fi drivers, network buffers, and RTOS queues. Ed25519 consumes only 108 bytes of dynamic heap during execution, whereas RSA-3072 allocates 4,732 bytes (a 44-fold increase), minimizing the risk of heap fragmentation.
- Deterministic Execution Timing:** Ed25519 operations execute in constant time, producing consistent latencies on every run. In contrast, RSA key generation times fluctuate unpredictably between 210 ms and 16.2 seconds depending on prime search iterations, introducing unacceptable latency spikes into connection setup.
- Asymmetric Workload Trade-off:** Although RSA verifies signatures faster (1.66 ms versus 19.77 ms), an IoT VPN client must generate keys and sign challenges. RSA’s 6-second key generation latency and 768-byte packet size make it unviable for embedded client endpoints.

5 Serial Monitor Output Verification

The terminal captures below confirm execution runs and heap measurements recorded on the ESP32 hardware:

5.1 Ed25519 Serial Output

Figure 1 displays the serial output across 100 runs, showing 8.49 ms keygen, 9.35 ms signing, 19.77 ms verification, and 108 bytes heap delta.

```
-----
Fast tests: 100 runs | Slow tests: 10 runs
Free Heap at Start: 329528 bytes

[1/3] Testing Ed25519 (Fast and Modern)
Memory Before Test: 329528 bytes
Memory After Test : 329420 bytes
Memory Difference : 108 bytes

Creating Keys      : 2038136 cycles | 8.49 ms (Average over 100 runs)
Signing Message    : 2244252 cycles | 9.35 ms (Average over 100 runs)
Verifying Signature: 4744893 cycles | 19.77 ms (Average over 100 runs)

Public Key Size    : 32 bytes
Signature Size     : 64 bytes
```

Figure 1: ESP32 Serial Output for Ed25519 Benchmark (100 runs).

5.2 ECDSA P-256 Serial Output

Figure 2 displays the ECDSA P-256 serial output (145.94 ms keygen, 157.76 ms signing, 312.14 ms verification, 408 bytes heap delta).

```
[2/3] Testing ECDSA P-256 (Standard and Secure)
Memory Before Test: 328804 bytes
Memory After Test : 328396 bytes
Memory Difference : 408 bytes

Creating Keys      : 35025008 cycles | 145.94 ms (Average over 100 runs)
Signing Message    : 37861518 cycles | 157.76 ms (Average over 100 runs)
Verifying Signature: 74912688 cycles | 312.14 ms (Average over 100 runs)

Public Key Size    : 65 bytes (uncompressed)
Signature Size     : 70 bytes (DER, last sig)
```

Figure 2: ESP32 Serial Output for ECDSA P-256 Benchmark (100 runs).

5.3 RSA-3072 Serial Output

Figure 3 displays the 10 individual RSA keygen runs, subsequent signing/verification, and 4,732-byte heap allocation.

5.4 Execution Summary and Final Heap Status

Figure 4 shows the benchmark completion status and ending free heap (328,940 bytes).

```

[3/3] Testing RSA-3072 (Heavy and Classic)
Running RSA-3072 keygen (10 iterations, this takes time)...
- Keygen iteration 1/10 ... OK (5095 ms)
- Keygen iteration 2/10 ... OK (6125 ms)
- Keygen iteration 3/10 ... OK (4887 ms)
- Keygen iteration 4/10 ... OK (4956 ms)
- Keygen iteration 5/10 ... OK (13736 ms)
- Keygen iteration 6/10 ... OK (1175 ms)
- Keygen iteration 7/10 ... OK (5922 ms)
- Keygen iteration 8/10 ... OK (210 ms)
- Keygen iteration 9/10 ... OK (3056 ms)
- Keygen iteration 10/10 ... OK (16231 ms)
Generating persistent RSA-3072 key for sign/verify ...
Memory Before Test: 328696 bytes
Memory After Test : 323964 bytes
Memory Difference : 4732 bytes

Creating Keys      : 1473414175 cycles | 6139.23 ms (Average over 10 runs)
Signing Message   : 88830344 cycles | 370.13 ms (Average over 100 runs)
Verifying Signature: 399080 cycles | 1.66 ms (Average over 100 runs)

Public Key Size   : 384 bytes
Signature Size    : 384 bytes

```

Figure 3: ESP32 Serial Output for RSA-3072 Benchmark (10 keygen runs, 100 sign/verify runs).

```

Benchmark Finished!
Free Heap at End: 328940 bytes
You can now copy these results for your thesis.

```

Figure 4: ESP32 Serial Output showing completion and final free heap memory.

6 Graphical Benchmark Visualizations

The figures below plot the empirical metrics across execution latency, wire payload, memory usage, and stochastic variance:

6.1 Execution Latency Comparison

Figure 5 compares key generation, signing, and verification latency on a logarithmic scale.

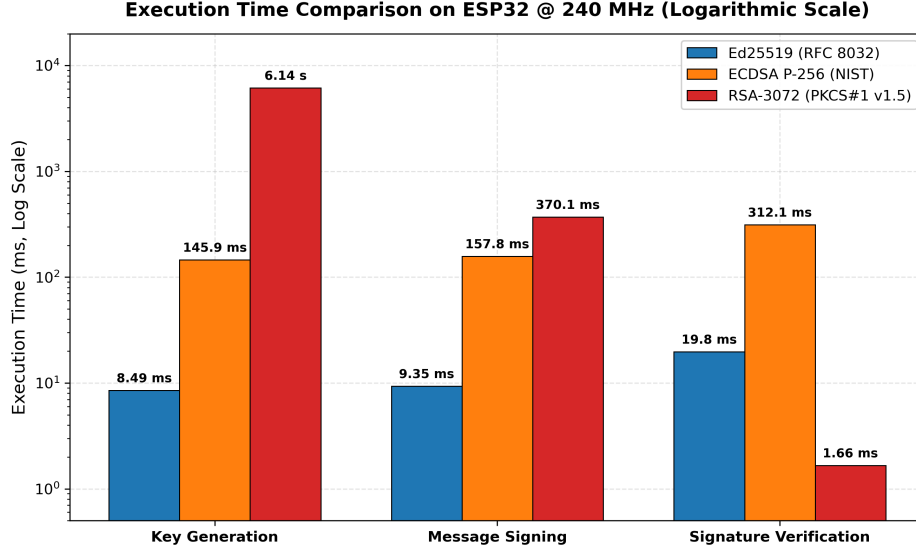


Figure 5: Execution time comparison (logarithmic scale) for KeyGen, Signing, and Verification.

6.2 Handshake Communication Overhead

Figure 6 breaks down public key and signature sizes transmitted per handshake round.

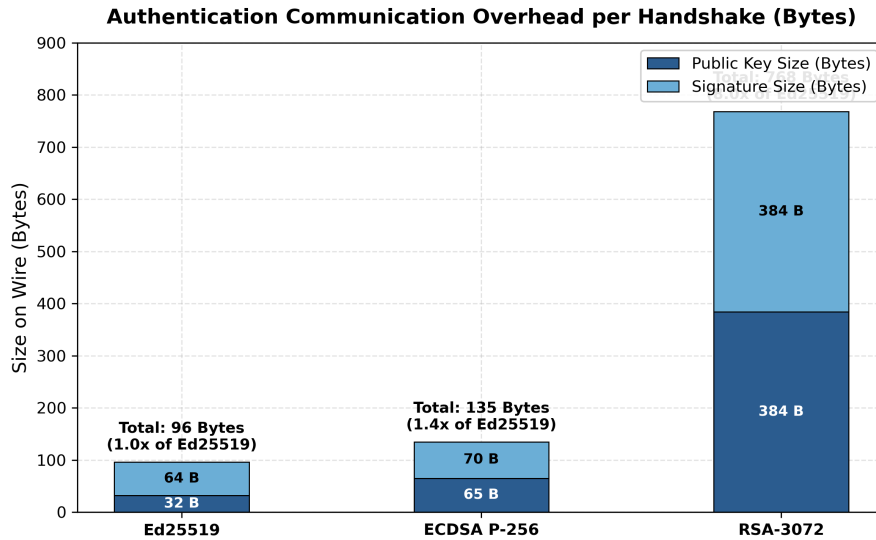


Figure 6: Authentication handshake overhead (Public Key + Signature size in bytes).

6.3 Dynamic Heap Memory Allocation

Figure 7 illustrates the peak heap RAM consumed during algorithm execution.

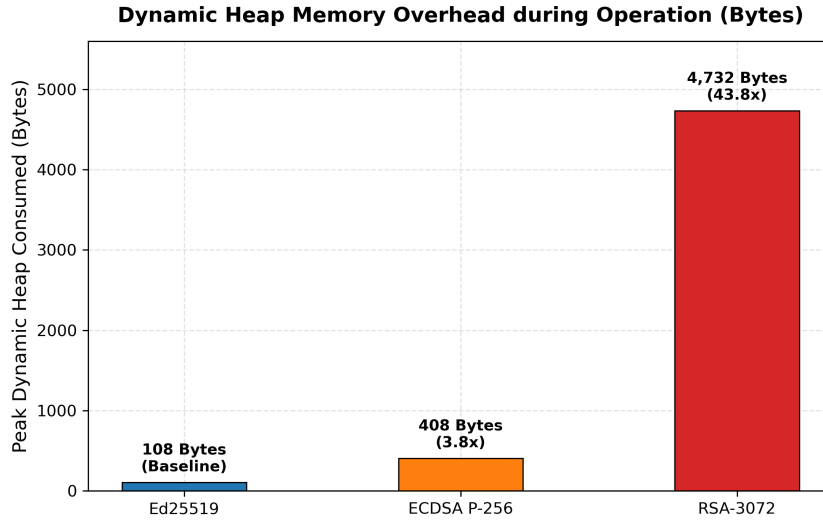


Figure 7: Dynamic heap memory consumed by each algorithm on the ESP32.

6.4 RSA-3072 Key Generation Variance

Figure 8 plots the individual latencies recorded across the 10 RSA key generation runs.

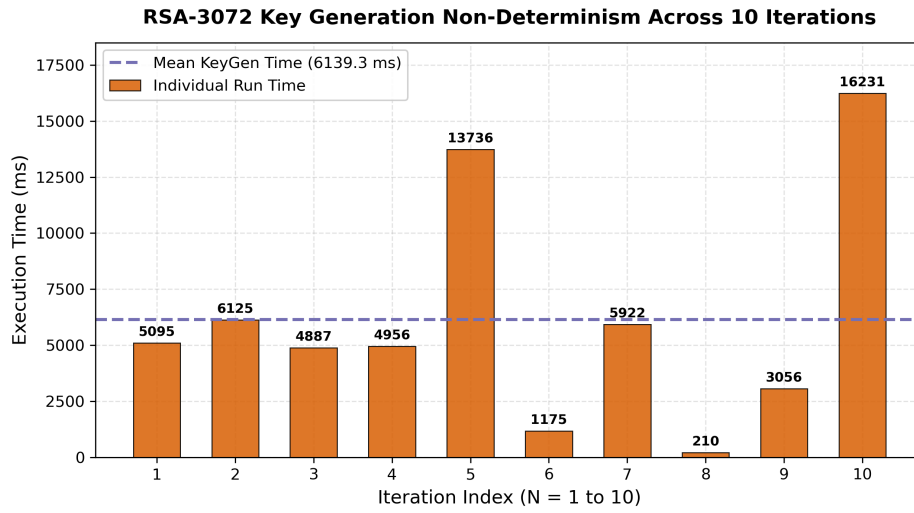


Figure 8: Unpredictability of RSA-3072 key generation across 10 runs (210 ms to 16,231 ms).

7 Benchmarking Logic and Pseudocode

The pseudocode below details the measurement routine implemented on the ESP32, including cycle counter acquisition, heap profiling, and test loops for each algorithm:

7.1 Step 1: Setup and Hardware Measurement Helpers

```
# Configuration: ESP32 runs at 240 MHz clock speed
CPU_SPEED_MHZ = 240
FAST_RUNS = 100          # Runs for fast algorithms
SLOW_RSA_RUNS = 10       # Runs for RSA keygen due to prime search latency

# Dummy 64-byte challenge payload simulating VPN nonce
test_challenge = generate_random_bytes(64)

# Reads hardware CPU cycle register directly from ESP32 core
function read_cpu_cycles():
    return hardware_register_read("ccount")

# Converts raw CPU clock cycles into milliseconds
function cycles_to_milliseconds(cycles):
    return cycles / (CPU_SPEED_MHZ * 1000)

# Computes arithmetic mean cycle count over N iterations
function calculate_average(cycle_array, run_count):
    total_cycles = sum(cycle_array)
    return total_cycles / run_count
```

7.2 Step 2: Ed25519 Benchmark Routine

```
# Step 2: Test Ed25519 (Fast and Modern)
memory_before = esp32_get_free_heap()

for i in range(1, FAST_RUNS):
    # 1. Key Generation
    start = read_cpu_cycles()
    public_key, private_key = ed25519_create_keypair()
    keygen_cycles[i] = read_cpu_cycles() - start

    # 2. Message Signing (64-byte challenge)
    start = read_cpu_cycles()
    signature = ed25519_sign_detached(test_challenge, private_key)
    sign_cycles[i] = read_cpu_cycles() - start

    # 3. Verification
    start = read_cpu_cycles()
    is_valid = ed25519_verify_signature(signature, test_challenge, public_key)
    verify_cycles[i] = read_cpu_cycles() - start

memory_after = esp32_get_free_heap()
memory_difference = memory_before - memory_after

# Compute final Ed25519 timing metrics
ed25519_avg_keygen_ms = cycles_to_milliseconds(calculate_average(keygen_cycles,
    FAST_RUNS))
ed25519_avg_sign_ms   = cycles_to_milliseconds(calculate_average(sign_cycles,
    FAST_RUNS))
```

```
ed25519_avg_verify_ms = cycles_to_milliseconds(calculate_average(verify_cycles,
    FAST_RUNS))
```

7.3 Step 3: ECDSA P-256 Benchmark Routine

```
# Step 3: Test ECDSA P-256 (Standard NIST Curve)
memory_before = esp32_get_free_heap()

# Pre-hash challenge payload using SHA-256
challenge_hash = sha256(test_challenge)

for i in range(1, FAST_RUNS):
    # 1. Key Generation
    start = read_cpu_cycles()
    ecdsa_keypair = ecdsa_p256_generate_keypair()
    keygen_cycles[i] = read_cpu_cycles() - start

    # 2. Message Signing
    start = read_cpu_cycles()
    signature = ecdsa_p256_sign_hash(challenge_hash, ecdsa_keypair.private_key)
    sign_cycles[i] = read_cpu_cycles() - start

    # 3. Verification
    start = read_cpu_cycles()
    is_valid = ecdsa_p256_verify_signature(signature, challenge_hash,
    ecdsa_keypair.public_key)
    verify_cycles[i] = read_cpu_cycles() - start

memory_after = esp32_get_free_heap()
memory_difference = memory_before - memory_after

# Compute final ECDSA timing metrics
ecdsa_avg_keygen_ms = cycles_to_milliseconds(calculate_average(keygen_cycles,
    FAST_RUNS))
ecdsa_avg_sign_ms = cycles_to_milliseconds(calculate_average(sign_cycles,
    FAST_RUNS))
ecdsa_avg_verify_ms = cycles_to_milliseconds(calculate_average(verify_cycles,
    FAST_RUNS))
```

7.4 Step 4: RSA-3072 Benchmark Routine

```
# Step 4: Test RSA-3072 (Heavy Factorization)

# Part A: Key Generation (10 iterations due to prime searching duration)
for i in range(1, SLOW_RSA_RUNS):
    start = read_cpu_cycles()
    # Search for two 1536-bit prime numbers with public exponent 65537
    rsa_keypair = rsa_3072_generate_keypair(exponent = 65537)
    rsa_keygen_cycles[i] = read_cpu_cycles() - start
    free_memory(rsa_keypair)

# Part B: Sign and Verify across 100 iterations using persistent keypair
persistent_rsa_key = rsa_3072_generate_keypair(exponent = 65537)
memory_before = esp32_get_free_heap()

for i in range(1, FAST_RUNS):
    # 1. Message Signing (PKCS#1 v1.5 with SHA-256)
```

```

    start = read_cpu_cycles()
    rsa_signature = rsa_3072_sign(challenge_hash, persistent_rsa_key.
private_key)
    rsa_sign_cycles[i] = read_cpu_cycles() - start

    # 2. Verification
    start = read_cpu_cycles()
    is_valid = rsa_3072_verify(rsa_signature, challenge_hash,
persistent_rsa_key.public_key)
    rsa_verify_cycles[i] = read_cpu_cycles() - start

memory_after = esp32_get_free_heap()
memory_difference = memory_before - memory_after

# Compute final RSA timing metrics
rsa_avg_keygen_ms = cycles_to_milliseconds(calculate_average(rsa_keygen_cycles,
    SLOW_RSA_RUNS))
rsa_avg_sign_ms    = cycles_to_milliseconds(calculate_average(rsa_sign_cycles,
    FAST_RUNS))
rsa_avg_verify_ms  = cycles_to_milliseconds(calculate_average(rsa_verify_cycles,
    FAST_RUNS))

```